

*****abstract

An abstract class in Java is a restricted class that cannot be instantiated directly and serves as a blueprint for other classes.

Declared using the abstract keyword, it achieves partial abstraction by allowing you to mix complete methods (with bodies) alongside incomplete methods (without bodies).

The Core Rules
No Direct Objects: You cannot use new on an abstract class (e.g., new Animal() throws a compiler error).

Abstract Methods: Methods marked abstract have a signature but no body.

Subclasses must override them.
Concrete Methods: It can contain fully implemented regular methods, which subclasses inherit automatically.

Constructors Allowed: It can contain constructors called during subclass instantiation to initialize common variables.

Variables Allowed: Unlike interfaces, it can contain any instance variables (private, protected, public).
Code Example: The Vehicle Blueprint
Consider a generic Vehicle. You never buy a "generic vehicle"—you buy a specific Car or Motorcycle. Therefore, Vehicle should be an abstract concept.
java// Abstract Class abstract class Vehicle { String brand;

```
// Constructor
public Vehicle(String brand) {
    this.brand = brand;
}

// Abstract Method: Every vehicle starts, but differently. No body {} here!
abstract void startEngine();

// Concrete Method: Every vehicle hoots the same way.
public void honk() {
    System.out.println("Beep beep!");
}

}
```

```
// Concrete Subclass (Must implement the abstract methods) class Car extends Vehicle {
public Car(String brand) { super(brand); // Calls the abstract class constructor }
}
```

```
@Override
void startEngine() {
    System.out.println(brand + " engine starts with a button press.");
}
```

}

```
// Execution Class public class Main { public static void main(String[] args) { Vehicle myVehicle = new Vehicle("Generic"); // ERROR: Cannot instantiate
```

```
Vehicle myCar = new Car("Tesla"); // Valid via polymorphism
myCar.startEngine(); // Prints: Tesla engine starts with a button press.
myCar.honk();          // Prints: Beep beep!
}
```

} Use code with caution.

Abstract Class vs. InterfaceWhile both are used for abstraction, they serve different design purposes: **يُمكن للتصديق بشكل لا حصر له**

Feature	Abstract Class	Interface
Abstraction Level	Partial abstraction (can contain both abstract and concrete methods)	Full abstraction (primarily abstract methods; can also contain default and static methods)
Inheritance	A subclass can extend only one abstract class	A class can implement multiple interfaces
Variables	Can have instance, static, final, and private fields	Fields are implicitly public, static, and final
Constructors	Can have constructors	Cannot have constructors
Methods	Can contain both abstract and implemented methods	Traditionally contains abstract methods; can also have default and static methods
Access Modifiers	Supports all access modifiers (private, protected, public, etc.)	Methods are public by default (except private methods allowed in newer Java versions)
Purpose	Used when classes share common state and behavior	Used to define a contract that multiple classes can implement

Summary

- **Abstract Class** → Used when related classes share common attributes and behavior.
- **Interface** → Used to define a contract that can be implemented by unrelated classes, supporting multiple inheritance of type.

*interface**

A java interface is a completely abstract blueprint or contract that specifies what a class

A Java interface is a completely abstract blueprint or contract that specifies what a class must do, but not how it does it.

It allows unrelated classes to implement the exact same behavior, forcing consistency across your code.

Why Use Interfaces?

Achieve Abstraction: They separate the design of an object from its actual implementation details.

Loose Coupling: Code can interact with the interface type rather than a specific class, making components easy to swap out.

Multiple Inheritance: Java classes can only inherit from one parent class, but they can implement multiple interfaces.

Key Rules of Interfaces

Instantiation: You cannot create an object directly from an interface using the new keyword.

Implicit Modifiers: Every regular method declared inside an interface is automatically public and abstract.

Implicit Constants: Every variable declared inside an interface is automatically public, static, and final.

No Constructors: Since interfaces cannot maintain state or be instantiated directly, they do not have constructors.

Code Example:

How It Works

Here is how you define a Vehicle interface and implement it across different classes using the implements keyword.

```
1. Define the Interface (The Contract)

interface Vehicle {
    // Implicitly public, static, and final constant
    int MAX_SPEED = 120;

    // Implicitly public and abstract method (no body)
    void startEngine();
}

// 2. First implementation class
class Car implements Vehicle {
    @Override public void startEngine() {
        System.out.println("Car engine started turning over key.");
    }
}

// 3. Second implementation class
class ElectricTruck implements Vehicle {
    @Override public void startEngine() {
        System.out.println("Electric truck powered on silently.");
    }
}
```

Use code with caution.

How to Use Interfaces in PracticeBecause of polymorphism, you can declare a variable using the interface type as its reference while pointing to any class object that implements it.

```
javapublic class Main {
```

```
    public static void main(String[] args) {  
        // Interface reference pointing to a Car object  
        Vehicle myVehicle = new Car();  
        myVehicle.startEngine(); // Prints: Car engine started...  
  
        // Reassigning reference to an ElectricTruck object  
        myVehicle = new ElectricTruck();  
        myVehicle.startEngine(); // Prints: Electric truck powered...  
    }  
}
```

} Use code with caution.

Modern Java Additions (Java 8 and 9+)While early versions of Java only allowed abstract methods in interfaces, modern Java allows:

Default Methods (Java 8+): Methods with bodies using the default keyword so you can expand interfaces without breaking existing implementing classes.

Static Methods (Java 8+): Methods called directly using the interface name (Vehicle.someStaticMethod()).

Private Methods (Java 9+): Used as internal helper functions shared inside default methods

Java Abstract Class vs Interface: When to Use

Use an Abstract Class When...	Use an Interface When...
Multiple related classes share common state (fields) and behavior.	You want to define a common contract that different classes must follow.
You need instance variables or constructors.	You need multiple inheritance of type (a class can implement multiple interfaces).
You want to provide some default implementation and force subclasses to implement specific methods.	Classes may be unrelated but should support the same behavior.
There is a strong "is-a" relationship between the parent and child classes.	You are describing a capability or role rather than a parent-child relationship.

Example: Abstract Class

```
abstract class Vehicle {
    protected String brand;

    public Vehicle(String brand) {
        this.brand = brand;
    }

    public void startEngine() {
        System.out.println("Engine started");
    }

    abstract void move();
}

class Car extends Vehicle {
    public Car(String brand) {
        super(brand);
    }

    @Override
    void move() {
        System.out.println("Car is moving");
    }
}
```

Why Abstract Class?

- All vehicles have a brand.
- All vehicles can start an engine.
- Some behavior is shared, while other behavior varies.

Example: Interface

```
interface Flyable {
    void fly();
}

class Bird implements Flyable {
    @Override
    public void fly() {
        System.out.println("Bird is flying");
    }
}

class Airplane implements Flyable {
    @Override
    public void fly() {
```

```
        System.out.println("Airplane is flying");
    }
}
```

Why Interface?

- Birds and airplanes are unrelated classes.
 - They both share the capability to fly.
 - A class can implement multiple interfaces.
-

Interview Answer (Short)

Use an **abstract class** when you want to share common state and behavior among closely related classes. Use an **interface** when you want to define a contract or capability that can be implemented by multiple unrelated classes, especially when multiple inheritance is needed.